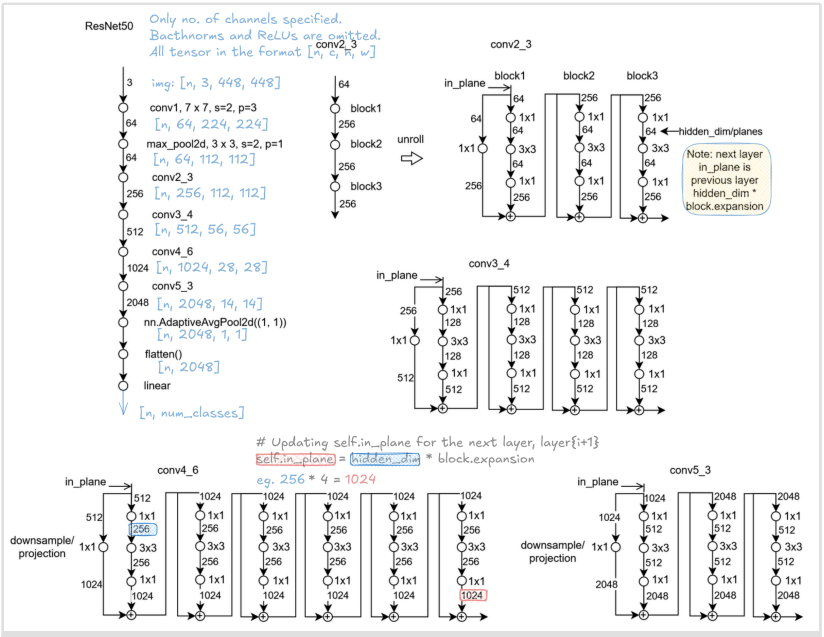
Image_Recognition

1. ResNet

```
import torchvision
from torchsummaryX import summary

 # model = torchvision.models.resnet18()
 model = torchvision.models.resnet50()
 # x = torch.zeros((10,3,224,224))  # output size before avgpool: 7 x 7
 x = torch.zeros((10, 3, 448, 448))  # output size before avgpool: 14 x 14
 summary(model, x)
```

2. ResNet50:



ResNet50

1. Code: modules/resnet/block.py

```
import torch
import torch.nn as nn


def conv3x3(input_dim, output_dim, stride=1):
    return nn.Conv2d(
        input_dim, output_dim, kernel_size=3, stride=stride, padding=1, bias=False
    )


def conv1x1(input_dim, output_dim, stride=1):
    return nn.Conv2d(
        input_dim, output_dim, kernel_size=1, stride=stride, padding=0, bias=False
    )


class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, input_dim, hidden_dim, stride=1, downsample=None):
        super().__init__()
        self.downsample = downsample
        self.residual = nn.Sequential(
            conv3x3(input_dim, hidden_dim, stride),
            nn.BatchNorm2d(hidden_dim),
            nn.ReLU(inplace=True),
            conv3x3(hidden_dim, hidden_dim * BasicBlock.expansion),
            nn.BatchNorm2d(hidden_dim * BasicBlock.expansion),
        )
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        residual = self.residual(x)
        if self.downsample is not None:
            skip_connection = self.downsample(x)
        else:
            skip_connection = x
        out = self.relu(residual + skip_connection)
        return out
```

```python
class BottleNeckBlock(nn.Module):
    expansion = 4

    def __init__(self, input_dim, hidden_dim, stride=1, downsample=None):
        super().__init__()
        self.downsample = downsample
        self.residual = nn.Sequential(
            conv1x1(input_dim, hidden_dim, stride),
            nn.BatchNorm2d(hidden_dim),
            nn.ReLU(inplace=True),
            conv3x3(hidden_dim, hidden_dim),
            nn.BatchNorm2d(hidden_dim),
            nn.ReLU(inplace=True),
            conv1x1(hidden_dim, hidden_dim * BottleNeckBlock.expansion),
            nn.BatchNorm2d(hidden_dim * BottleNeckBlock.expansion),
        )
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        residual = self.residual(x)
        if self.downsample is not None:
            skip_connection = self.downsample(x)
        else:
            skip_connection = x
        # print('residual.shape', residual.shape)
        # print('skip_connection.shape', skip_connection.shape)
        out = self.relu(residual + skip_connection)
        return out
```

2. Code: model/resnet.py

```python
import torch.utils.model_zoo as model_zoo

from modules.resnet.block import *


class ResNet(nn.Module):

    def __init__(self, block, num_blocks, num_classes):
        super().__init__()

        # Conv_1 x 1
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)

        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.in_plane = 64
        self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)  # conv_2
        self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)  # conv_3
        self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)  # conv_4
        self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)  # conv_5

        self.cls_head = self._make_cls_head(512 * block.expansion, num_classes)

        def _weights_init(m):
            """kaiming init (https://arxiv.org/abs/1502.01852v1)"""
            if isinstance(m, nn.Linear) or isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight)
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)

        self.apply(_weights_init)

    def _make_layer(self, block, hidden_dim, num_blocks, stride=1):
        downsample = None
        if stride != 1 or hidden_dim * block.expansion != self.in_plane:
            downsample = nn.Sequential(
                nn.Conv2d(
                    self.in_plane,
                    hidden_dim * block.expansion,
                    kernel_size=1,
                    stride=stride,
                    bias=False,
                ),
                nn.BatchNorm2d(hidden_dim * block.expansion),
            )
        layers = []
        for i in range(num_blocks):
            if i == 0:
                layers.append(block(self.in_plane, hidden_dim, stride, downsample))
                self.in_plane = hidden_dim * block.expansion
            else:
                layers.append(block(self.in_plane, hidden_dim))

        return nn.Sequential(*layers)
```

```
    def _make_cls_head(self, input_dim, num_classes):
        """cls: number of classes"""
        layers = [
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(input_dim, num_classes),
        ]
        return nn.Sequential(*layers)

    def forward(self, x):
        c1 = self.conv1(x)
        c1 = self.bn1(c1)
        c1 = self.relu(c1)

        c1 = self.maxpool(c1)

        c2 = self.layer1(c1)
        c3 = self.layer2(c2)
        c4 = self.layer3(c3)
        c5 = self.layer4(c4)

        out = self.cls_head(c5)

        return out


"""
    address to load pretrained model
"""
model_urls = {
    "resnet18": "https://download.pytorch.org/models/resnet18-5c106cde.pth",
    "resnet34": "https://download.pytorch.org/models/resnet34-333f7ec4.pth",
    "resnet50": "https://download.pytorch.org/models/resnet50-19c8e357.pth",
    "resnet101": "https://download.pytorch.org/models/resnet101-5d3b4d8f.pth",
}


def resnet18(num_classes, pretrained=False):
    """Constructs a ResNet-18 model.
    Args:
        pretrained (bool): If True, returns a model pre-trained on ImageNet
    """
    resnet_state_dict = model_zoo.load_url(model_urls["resnet18"])

    model = ResNet(BasicBlock, [2, 2, 2, 2], num_classes)
    if pretrained:
        print("Loading pretrained ...")
        model_state_dict = model.state_dict()
        for k in resnet_state_dict.keys():
            if k in model_state_dict.keys() and not k.startswith("fc"):
                model_state_dict[k] = resnet_state_dict[k]
        model.load_state_dict(model_state_dict)

    return model


def resnet34(num_classes, pretrained=False):
    """Constructs a ResNet-34 model.
    Args:
        pretrained (bool): If True, returns a model pre-trained on ImageNet
    """
    resnet_state_dict = model_zoo.load_url(model_urls["resnet34"])

    model = ResNet(BasicBlock, [3, 4, 6, 3], num_classes)
    if pretrained:
        print("Loading pretrained ...")
        model_state_dict = model.state_dict()
        for k in resnet_state_dict.keys():
            if k in model_state_dict.keys() and not k.startswith("fc"):
                model_state_dict[k] = resnet_state_dict[k]
        model.load_state_dict(model_state_dict)

    return model


def resnet50(num_classes, pretrained=False):
    """Constructs a ResNet-50 model.
    Args:
        pretrained (bool): If True, returns a model pre-trained on ImageNet
    """
    resnet_state_dict = model_zoo.load_url(model_urls["resnet50"])

    model = ResNet(BottleNeckBlock, [3, 4, 6, 3], num_classes)
    if pretrained:
        print("Loading pretrained ...")
        model_state_dict = model.state_dict()
        for k in resnet_state_dict.keys():
            if k in model_state_dict.keys() and not k.startswith("fc"):
                model_state_dict[k] = resnet_state_dict[k]
        model.load_state_dict(model_state_dict)
```

```
    return model


def resnet101(num_classes, pretrained=False):
    """Constructs a ResNet-101 model.
    Args:
        pretrained (bool): If True, returns a model pre-trained on ImageNet
    """
    resnet_state_dict = model_zoo.load_url(model_urls["resnet101"])

    model = ResNet(BottleNeckBlock, [3, 4, 23, 3], num_classes)
    if pretrained:
        print("Loading pretrained ...")
        model_state_dict = model.state_dict()
        for k in resnet_state_dict.keys():
            if k in model_state_dict.keys() and not k.startswith("fc"):
                model_state_dict[k] = resnet_state_dict[k]
        model.load_state_dict(model_state_dict)

    return model
```

3. Squeeze Excitation Block
    1. Consists of the following steps:
        1. Squeeze, Global Average pooling: nn.AdaptiveAvgPool(1)
        2. Excite, scale x tensor
        3. Transform (MLP), MLP=(Linear+NonlinearActivation) x N
    2. Is used in EfficientNet and EfficientNetV2 families.

```
class SqueezeExcitation(nn.Module):
    def __init__(self, input_chanels, squeeze_channels):
        super().__init__()

        self.squeeze = torch.nn.AdaptiveAvgPool2d(1)  # GAP layer
        self.excitation = nn.Sequential(
            nn.Flatten(),
            nn.Linear(input_channels, squeeze_channels),
            nn.ReLU(),
            nn.Linear(squeeze_channels, input_channels),
            nn.Sigmoid()  # squeeze the weights between 0 and 1
        )
```

4. Freezing layers and parameters

```
for param in model.parameters():
    param.requires_grad = False
```

Similarly, to freeze the parameters of a single layer. For example, say this layer is called f_c, then:

```
for param in model.fc.parameters():
    param.requires_grad = False
```

we can thaw parameters that are frozen by setting requires_grad = True.

- The BatchNorm layer is a special case: it has two parameters (gamma and beta), but it also has two buffers that are used to accumulate the mean and standard deviation of the dataset during training. If we only use requires_grad=False then we are only fixing gamma and beta. The statistics about the dataset are still accumulated. Sometimes fixing those as well can help the performance, but not always. Experimentation as usual is key.
- If we want to also freeze the statistics accumulated we need to put the entire layer in evaluation mode by using eval (instead of requires_grad=False for its parameters):

```
model.bn.eval()
```

- Note that this is different than using model.eval() (which would put the entire model in evaluation mode). You can invert this operation by putting the BatchNorm layer back into training mode: model.bn.train().

5. ViT
    1. Uses only the encoder part to use the bidirectional attention part to learn the whole image.
    2. Given a $3 \times 256 \times 256$ image, instead of directly using all the pixels to attend to each other which will make a $O((256 * 256)^2)$ which will be computationally expensive, the paper suggests to divide the image into patches of size $3 \times 16 \times 16$. Then flattening each patch into a 768 vector, let it through a linear layer to match the input size to the transformer encoder to be a 1024 vector, and adding positional embeddings also of size a 1024 vector. Resulting in $16 \times 16 = 256$ vectors of size 1024 each.
    3. CLS token, borrowed from BERT to do image classification.
    4. Probing Vision Transformer [1]
        1. Mean attention distance

        1. lower layers tend to focus on a mixture of local + global context.
        2. higher layers tend to focus on global contexts.
        3. MAD = f(data, depth), strong connection between MAD, pre-training data, ViT architecture.
        4. Without enough data, lower layers in ViT don't learn locality. This becomes evident in deeper architectures.
        5. with enough pre-training data, lower layers learn to encode locality early on which could be an indicator for good performance.
        6. ViT layers have access to global information almost uniformly. What are its repercussions?
        7. There's a primary difference here -- CNN's don't combine both global and local information like ViTs do.
        8. Does this lead to differences in their representation space? Yes, it does.
    2. Centered kernel alignment (CKA): A quantifiable way to measure the differences between representations in neural architectures.
        1. Invariant to orthogonal transformations of representations: even though we permute the order of the neurons, the CKA will remain invariant to that.
        2. Invariant to isortropic scaling: when we scale each dimension of each dimension uniformly, it still remains invariant.
    3. Role of skip connections
        1. ViT's representation space is uniform.
        2. Information from lower layer is propagated to higher layers more faithfully.
        3. How? The setup -- plot norm ratio of the following $||z_i||/||f(z_i)||$. z_i is the hidden representations of the ith layer (skip connection), $f(z_i)$ is transformation of z_i from the long branch (MLP or self-attention).
        4. This indicates that skip-connections play a much more influential role in modeling and propagating the repr across different layers if ViTs than in ResNets.
        5. If we remove skip connections in transformer blockes esp. in higher end, then the uniformity gets disrupted much more than the lower end.
        6. ViT's uniform representation structure (uniformity) affects robustness, against corruption, towards natural adverserial examples and so on. [2]
5. Explaining Transformers:
6. Attention as explanation:
    1. Separation of the CLS token and spatial tokens.
    2. Role of pre-training in the development of saliency.
    3. Revisit SA block from ViT
        1. Inputs: CLS token + Image patch tokens.
        2. Responsible for:
            1. Learning dependencies between the image tokens.
            2. Summarizing info into a CLS token for the head.
        3. What if it could be better separated? Delegating responsibilities better -- CLass attention.
7. from [4]:
    1. Assumptions:
        1. Input image size: H: 224 x W: 224 x C: 3
        2. Patch size: p1: 16 x p2: 16
        3. Embedding dim: embed_dim: $p1 * p2 * C$: 768
        4. Number of patches: num_patches: $H/p1$: 14 x $W/p2$: 14 = 196
        5. Number of transformer layers: num_layers: 12
        6. MLP hidden size: ffn_dim: 3072
        7. Output classes: num_classes: 1000
    2. Input Image
        1. Input: $[\mathrm{B}, C, H, W]$
        2. No transformation yet
    3. Patch embedding
        1. Image split into 196 patches of size 16 x16 x 3
        2. Flattened and linearly projected
        3. Output: $[\mathrm{B}, num\_patches, embed\_dim]$
    4. Add [CLS] Token
        1. Add learnable [CLS] token is prepended

    2. Output: $[\mathrm{B}, num\_patches + 1, embed\_dim]$
5. Position Encoding
    1. Add learnable/fixed positional embeddings to the sequence
    2. Output: $[\mathrm{B}, num\_patches + 1, embed\_dim]$
6. Transformer Encoder Block (repeated 12x)
    1. Multi-Head Attention
        1. Input: $[B, num\_patches + 1, embed\_dim]$
        2. Q, K, V: $[B, num\_patches + 1, embed\_dim] \rightarrow 12 \text{ heads} \rightarrow [B, num\_patches + 1, embed\_dim]$
        3. Output: $[\mathrm{B}, num\_patches + 1, embed\_dim]$
    2. MLP (Feedforward)
        1. Layer 1: $[B, num\_patches + 1, embed\_dim] \rightarrow [B, num\_patches + 1, ffn\_dim]$
        2. GELU activation
        3. Layer 2: $[B, num\_patches + 1, ffn\_dim] \rightarrow [B, num\_patches + 1, embed\_dim]$
        4. Output: $[B, num\_patches + 1, embed\_dim]$
    3. The shape remains $[B, num\_patches + 1, embed\_dim]$ through all 12 layers.
7. Extract[CLS] Token
    1. Select first token in sequence
    2. Output: $[B, embed\_dim]$
8. Classification Head
    1. Linear layer: $[B, embed\_dim] \rightarrow [B, num\_classes]$
    2. Output: $[B, num\_classes]$

References:

1. All things ViTs, CVPR 2023 Tutorial, Hila Chefer and Sayak Paul.
2. Vision Transformers and Robust learners.
3. MLP Mixer (MLP is all you need for vision).
4. https://hackmd.io/4nUSPFItRx-s4HERJzrliw?view